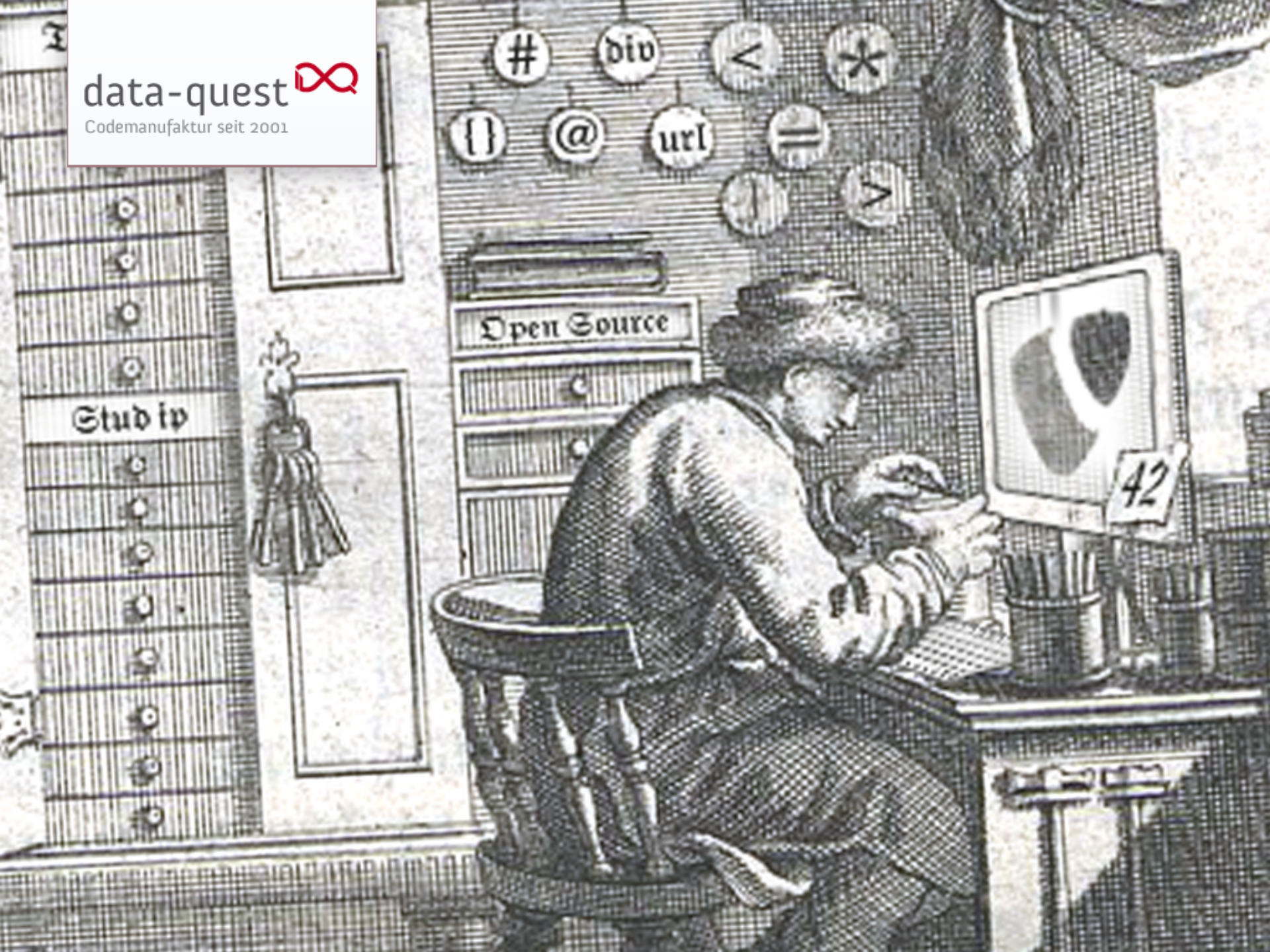






Attack of the S.O.R.M.

Entwicklertagung 2014 Passau, Donnerstag 20.03.2014

André Noack, data-quest GmbH

Schlachtplan

- Magische Angriffe
- in Reih und Glied: SimpleCollection
- Strategisches
- Manöverkritik
- Nach der Schlacht ist vor der Schlacht

Magische Angriffe

- `$sorm->id` ist immer der Primärschlüssel
 - mehrwertige werden aneinander gehängt
- `$sorm->getId()` gibt aber ein array bei mehrwertigen
- Reihenfolge von `SHOW COLUMNS FROM...`
- `autoincrement` wird erkannt
- ohne `autoincrement` wird neue Stud.IP-Style ID erzeugt

```
$foo = new SormWithAutoincrement();  
$foo->store();  
// $foo->id ist 1
```

```
$foo = new SormWithoutAutoincrement();  
$foo->store();  
// $foo->id ist a07535cf2f8a72df33c12ddfa4b53dde
```

Magische Angriffe

- `$sorm->name` ruft `$sorm->getValue('name')` auf
- wenn `$sorm->getName()` existiert, dann dieses
- analog dazu `setName("")`
- `additional_fields` funktionieren genauso, können aber auch explizit andere getter/setter haben
- `additional_fields` sind auch bei `toArray()` dabei
- `additional_fields` mit setter werden auch bei `setData()` befüllt

Magische Angriffe

```
class MightySormPower extends SimpleORMap
{
  private $bar;
  function __construct($id = null)
  {
    $this->additional_fields['bar'] = true;
    $this->additional_fields['foobar']['set'] = 'setBar';
    parent::__construct($id);
  }
  function getBar()
  {
    return $this->bar;
  }
  function setBar($bar)
  {
    $this->bar = $bar;
  }
}

$sorm = new MightySormPower();
$sorm->bar = 'foo';
$sorm->getBar(); // returns 'foo'
$sorm->toArray(); // returns inter alia array('bar' => 'foo')
$sorm->setData(array('foobar' => 'foobar'));
$sorm->getBar(); // returns 'foobar'
```

Magische Angriffe

- `$sorm->toArray(array('id', 'name'))` gibt nur angegebene Felder zurück
- `$sorm->toArrayRecursive()` kann das auch mit verschachtelten Arrays:
- `$sorm->toArrayRecursive(array('id','name','children' => array('id','name')))`
- `mkdate`, `chdate` wird erkannt und befüllt
- `::toObject()` gibt immer ein Objekt zurück, Parameter kann bereits ein Objekt sein, eine ID oder ein Array mit das auch das Schlüsselfeld enthält

Magische Angriffe

- ab 3.0: notification_map
 - Stud.IP-Notifications automatisch an SORM callbacks

```
$this->notification_map['after_create'] = 'UserDidCreate';  
$this->notification_map['after_store'] = 'UserDidUpdate';  
$this->notification_map['after_delete'] = 'UserDidDelete';  
$this->notification_map['before_create'] = 'UserWillCreate';  
$this->notification_map['before_store'] = 'UserWillUpdate';  
$this->notification_map['before_delete'] = 'UserWillDelete';
```


in Reih und Glied

- alle Relationen sind SimpleORMapCollection
- d.h. SimpleCollection mit der Erweiterung nur SORM Objekte aufzunehmen
- SimpleCollection speichert ArrayObject, übergebene Arrays werden umgewandelt
- gedacht für “result-set” artige Arrays
- Zugriff auf Elemente in [] oder -> Notation
- ist kein Array! (array Funktionen klappen nicht)

'foreign_key'

in Reih und Glied

- bietet `::each()` `::map()` mit einem Closure
- `::filter()` mit Closure liefert eine neue Collection
- übliche Filter über `::findBy()`, z.B. `== != > <`
- `::orderBy()` und `::limit()` wie in SQL
- `::first()` und `::val()`
- `::toGroupedArray()` für Aggregate
- `SimpleORMapCollection` hat zusätzlich ein `::find()` auf den Primärschlüssel
- und eine `::sendMessage()` um Methoden in allen enthaltenen Objekten aufzurufen

Strategisches

- Klassenname Singular
- `has_one` oder `belongs_to`? Wie sind die Besitzverhältnisse?
- `'class_name'` ist die einzige Pflichtangabe
- `'foreign_key'` default ist Primärschlüssel
- `'assoc_foreign_key'` default ist der Name des ersten Teils des Schlüssels (bei `belongs_to` einfach `'id'`)
- `'assoc_func'` default ist `::findby{assoc_foreign_key}`
- man kann aber eine Methode angeben, oder ein Closure

Strategisches

- 'on_delete' 'on_store' regeln, ob kaskadierend gespeichert oder gelöscht werden soll
- kann auch ein Closure sein
- 'additional_fields' 'get' und 'set' kann ein Closure oder der Name einer Methode sein
- 'default_values' sind bei vielen Stud.IP Tabellen nötig
- besser callbacks als Methoden überschreiben

Manöverkritik

- exzessiver Speicherverbrauch
- $n+1$ Problem
- serialisieren u.U. nicht möglich

Das wars...

Nach der Schlacht ist vor
der Schlacht!

Wünsche?